Object_Representation_ChatGPT

May 21, 2026

1 Object Representation

ChatGPT-5.5 Plus Prompt:

Suppose my object is a voice signal. It could be represented as a time sequence or as a Fourier transform approximation or as a spectrogram. Do I store all three representations in my signal object or do I store whichever representation comes first and then convert as necessary. How does my object container know which representation it already has and which it needs to generate?

This is exactly the right OOP design question: the issue is not "where do I put arrays?" but "which representation is authoritative, which ones are derived, and how do I keep them consistent?"

For a voice signal, I would usually store **one authoritative representation** and treat the others as **derived cached representations**.

Most often:

```
VoiceSignal
    authoritative:
        time-domain samples
        sampling rate

    derived:
        Fourier transform
        spectrogram
        features
```

1.1 Why time samples should usually be primary

The time-domain waveform is closest to the raw measured data:

```
x[n]
fs
```

From that you can compute:

```
FFT
spectrogram
MFCCs
pitch estimate
energy envelope
```

But the reverse is not always exact. A spectrogram often discards phase or depends on window length, overlap, scaling, etc. So it is better to treat the time signal as the "source of truth."

1.2 A good design pattern: lazy computation with caching

The object computes derived representations only when requested.

```
class VoiceSignal:

    def __init__(self, samples, fs):
        self.samples = samples
        self.fs = fs

        self._fft = None
        self._spectrogram = None

    def fft(self):
        if self._fft is None:
            print("Computing FFT...")
            self._fft = np.fft.rfft(self.samples)
        return self._fft

    def spectrogram(self):
        if self._spectrogram is None:
            print("Computing spectrogram...")
            self._spectrogram = compute_spectrogram(self.samples, self.fs)
        return self._spectrogram
```

So:

```
voice = VoiceSignal(samples, fs)

voice.fft()          # computed now
voice.fft()          # reused

voice.spectrogram()  # computed only when needed
```

The object "knows" what it has because cached fields start as `None`.

1.3 But you must handle invalidation

Suppose you modify the time signal:

```
voice.samples = filtered_samples
```

Then the old FFT and spectrogram are no longer valid.

So you need a method that updates the waveform and clears derived data:

```
class VoiceSignal:

    def __init__(self, samples, fs):
        self._samples = samples
        self.fs = fs
        self._fft = None
        self._spectrogram = None

    @property
    def samples(self):
        return self._samples

    def update_samples(self, new_samples):
        self._samples = new_samples
        self._fft = None
        self._spectrogram = None
```

Now the object stays consistent.

1.4 Better version: keep parameters with cached results

A spectrogram depends on choices:

```
window_length
overlap
window_type
nfft
```

So you may want a cache dictionary:

```
class VoiceSignal:

    def __init__(self, samples, fs):
        self.samples = samples
        self.fs = fs
        self._spectrogram_cache = {}

    def spectrogram(self, nperseg=256, noverlap=128):

        key = (nperseg, noverlap)

        if key not in self._spectrogram_cache:
            self._spectrogram_cache[key] = compute_spectrogram(
                self.samples,
                self.fs,
                nperseg=nperseg,
                noverlap=noverlap
            )
```

```
        return self._spectrogram_cache[key]
```

This way:

```
voice.spectrogram(256,128)
voice.spectrogram(512,256)
```

are treated as different derived representations.

1.5 The design rule

Use this hierarchy:

```
Raw data
    ↓
Primary object state
    ↓
Derived representations
    ↓
Cached results
```

For a voice signal:

```
samples + fs
    ↓
FFT
spectrogram
pitch contour
formants
MFCCs
```

Do **not** store all three immediately unless:

1. They are expensive to recompute,
2. You know they will all be used,
3. Memory is not a concern,
4. Their parameters are fixed and well-defined.

Otherwise, compute lazily.

1.6 Good mental model

The object should answer:

```
voice.time()
voice.fft()
voice.spectrogram()
```

but internally it should think:

```
Do I already have this representation?
    yes → return cached version
    no  → compute it, store it, return it
```

That is one of the most useful OOP patterns for scientific Python:

> **Store the source of truth. Lazily compute derived views. Cache them. Invalidate them when the source changes.**

[ ]: